Plan-First Parallelism for Hard Mathematical Reasoning: A Search-Space Failure Analysis*

Yuji Lim[1], Jeongsu Park[2], and Moohong Min[3†]

[1] Department of Computer Education, Sungkyunkwan University, Seoul, 03063, Republic of Korea
limuzi@g.skku.edu

[2] Department of Computer Education/Data Science, Sungkyunkwan University, Seoul, 03063, Republic of Korea
bakjs0422@g.skku.edu

[3] Department of Computer Education/Social Innovation Convergence Program, Sungkyunkwan University, Seoul, 03063, Republic of Korea
iceo@skku.edu

Abstract

Recent advances in large language model (LLM) research have prompted growing interest in improving reasoning capabilities, yet hard reasoning problems remain stubbornly difficult to solve. Evidence suggests that simply increasing test-time compute yields diminishing returns on the hardest problem regimes, and that the structure of the search space itself is the more critical factor. In this work, we propose Plan-First Parallelism (PF), a strategy-level branching approach that generates multiple distinct solution plans before execution, as opposed to answer-level parallel sampling. To analyze failure systematically, we introduce a three-way error taxonomy distinguishing search-space failure (E1), selection failure (E2), and plan-execution divergence (E3). Experiments on AIME and BeyondAIME benchmarks show that PF consistently outperforms Answer-First Parallelism (AF) for o3-mini, improving accuracy by +11.5%p on AIME and +14.5%p on BeyondAIME. However, PF fails to generalize across all models, and our analysis reveals that its effectiveness depends on the combination of reasoning quality and instruction-following capability. Beyond benchmark performance, our findings provide a failure-aware perspective on reliable and cyber-resilient LLM reasoning components for AI-enabled e-business information systems.

1 Introduction

Recent advances in large language model (LLM) research have highlighted the importance of test-time compute scaling as a complementary approach to increasing model parameter size. In particular, search-oriented inference strategies such as parallel sampling [4], verifier-guided search [16, 17], and tree-style exploration [6, 20, 21] have been shown to outperform simple sequential revision [14, 15] on challenging reasoning tasks. However, evidence also suggests that merely increasing the amount of computation does not guarantee consistent performance gains in the hardest problem regimes, and that the structure of the search process itself may be the more critical factor.

This work departs from that observation. We hypothesize that failures on hard reasoning problems stem not simply from insufficient computation, but from a more fundamental limitation: the model's inability to generate correct high-level solution strategies in the first place—a

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 33, July 8–10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).
†Corresponding author

phenomenon we term *search-space failure*. In other words, the primary bottleneck may lie not in how many candidate answers are produced, but in whether the model can branch into meaningfully distinct regions of the strategy space.

Within the broader landscape of search-oriented inference, we focus on one concrete and controllable instantiation: Plan-First Parallelism. Rather than addressing search in its full generality, our interest is in analyzing how strategy-level branching, as opposed to answer-level parallelism, alters the candidate generation distribution and the structure of failure.

The main contributions of this paper are as follows. First, we propose a novel error taxonomy that decomposes hard reasoning failures into three distinct categories: search-space failure (E1), selection failure (E2), and plan-execution divergence (E3). Second, we empirically compare Answer-First and Plan-First approaches across three models on the AIME [25] and BeyondAIME [26] benchmarks, demonstrating that Plan-First improves both candidate hit rate and final accuracy for reasoning-capable models such as o3-mini.[1] Third, we quantitatively measure the gap between plan diversity and execution diversity, providing insight into the remaining limitations of Plan-First and directions for future improvement.

From a systems perspective, Plan-First Parallelism also offers structural resilience: by maintaining multiple strategically distinct solution paths, the system remains robust to the failure of any single reasoning trajectory—a property relevant to the design of reliable AI-driven decision support systems.

Beyond mathematical reasoning itself, this study is motivated by the growing use of LLMs as reasoning components in AI-enabled e-business information systems. Such systems increasingly rely on LLMs for decision support, workflow assistance, risk analysis, and knowledge-intensive operations. In these contexts, reasoning failures are not merely isolated model errors; they may propagate into downstream organizational decisions and reduce the reliability of the overall information system. Therefore, understanding whether failures arise from candidate-generation failure, selection failure, or plan-execution divergence is important for designing reliable and cyber-resilient AI reasoning pipelines [27].

We use hard mathematical reasoning as a controlled diagnostic setting rather than as an end application. Exact-answer benchmarks allow reasoning failures to be measured objectively and decomposed systematically. From this perspective, Plan-First Parallelism and the proposed E1–E3 taxonomy provide a failure-aware analysis framework that can inform the design of auditable and recoverable LLM-enabled decision support systems.

2 Background and Related Work

Mathematical reasoning has long served as a challenging testbed for evaluating the capabilities of large language models [10, 19]. The MATH dataset [1] demonstrated that even large-scale models struggle with competition-level problems, and that accuracy remains low as problem difficulty increases. Building on this, recent work [2] showed that scaling test-time compute can improve performance on easier problems, but that this effect diminishes on harder reasoning tasks, where simply allocating more computation does not guarantee consistent gains. This suggests that the bottleneck on hard problems lies not in the quantity of compute, but in the structure of the search process itself.

One promising direction for restructuring this search process is through prompting strategies. Chain-of-thought (CoT) prompting [3] demonstrated that eliciting intermediate reasoning steps before a final answer substantially improves performance on multi-step reasoning tasks.

[1] Specifically, we use `o3-mini-2025-01-31` in all experiments.

Zero-shot CoT [9] further showed that appending a simple trigger phrase elicits step-by-step reasoning without any demonstrations, while Auto-CoT [12] proposed automatic construction of CoT demonstrations to reduce manual effort. Self-consistency [4] extended this by sampling multiple reasoning paths and selecting the most frequent answer to further boost accuracy. Plan-and-Solve prompting [5] explicitly instructs the model to devise a solution plan before execution, reducing missing-step errors in zero-shot settings. In a related vein, Tree of Thoughts [6] frames problem solving as a search over multiple reasoning paths with explicit evaluation at each step. We adopt this plan-first paradigm as the core instantiation of strategy-level branching in our framework.

A broader landscape of related methods addresses reasoning improvement from complementary angles. Decomposition-based approaches [7, 13] break problems into simpler subproblems, while program-aided reasoning [8] leverages code execution for precise computation. Reasoning-acting frameworks [22] extend planning to interactive settings. Self-improvement methods [11, 14, 15] explore iterative refinement as a complementary direction to search-based approaches. Step-level verification [16, 17] addresses answer selection through process supervision, offering an orthogonal lever for improving final accuracy. These advances build on the emergent reasoning capabilities of large-scale language models and rely on instruction-following abilities developed through alignment training [18, 23]. Beyond performance, the structural property of maintaining multiple distinct solution paths also relates to the notion of reasoning resilience—the ability of a system to remain robust when individual reasoning trajectories fail.

3 Problem Formulation

This section defines the basic setup and notation for mathematical reasoning problems, and presents an error taxonomy for analyzing hard reasoning failures.

In LLM-based mathematical reasoning, failures on hard problems may stem not from insufficient computation, but from the inability to explore correct solution strategies in the first place. Generating multiple answers to a hard reasoning problem does not guarantee diversity in solution strategies. Therefore, it is important to first explore different solution strategies. From this perspective, this study decomposes failures into three reported categories: search-space failure, selection failure, and plan-execution divergence.

We denote a mathematical reasoning problem as q and its ground truth answer as $y^*(q)$. The language model p_θ with parameters θ takes problem q as input and generates K candidate solutions $a_1, \ldots, a_K$. Here, z_i denotes the i-th solution plan, and a_i denotes the i-th candidate solution generated conditioned on that plan. The selector s is a function that selects the final answer $\hat{y}(q)$ from the generated candidates.

This study argues that final accuracy alone is insufficient to fully explain the failure structure, and therefore measures two metrics together. First, **CandHit** is a metric that identifies whether at least one correct answer exists among the K candidates:

$$\text{CandHit} = \frac{1}{|D|} \sum_{q \in D} \mathbf{1}\big[\exists\, i : \text{ans}(a_i) = y^*(q)\big]$$

In contrast, **Acc** is a metric that verifies whether the finally selected answer is correct:

$$\text{Acc} = \frac{1}{|D|} \sum_{q \in D} \mathbf{1}\big[\hat{y}(q) = y^*(q)\big]$$

CandHit is the key metric for the search stage, while Acc is an end-to-end performance metric. By examining both metrics together, it is possible to distinguish whether a failure occurred at

the search stage or the selection stage. Specifically, this study classifies incorrect answers into the following three types.

E1 (Search-space failure) refers to cases where CandHit = 0, that is, no correct candidate was generated. This represents a failure at the search stage.

E2 (Selection failure) refers to cases where CandHit = 1 but Acc = 0, that is, a correct candidate existed but the selector failed to choose it. This represents a bottleneck at the selection stage.

E3 (Plan-execution divergence) refers to a subtype of search-space failure in which diverse plans are generated but the resulting solution executions converge to similar incorrect trajectories. Operationally, E3 is identified among cases with CandHit = 0 when the Plan Diversity evaluated by LLM-as-a-Judge [24] is ≥ 4 and the Execution Diversity is ≤ 2. The thresholds $(4, 2)$ are heuristic and exploratory. A threshold sensitivity analysis is reported in Table 4.

Although E3 is conceptually a subtype of E1, we report E1, E2, and E3 as mutually exclusive categories in the result tables for interpretability. In the reported breakdowns, E1 denotes search-space failures that do not satisfy the E3 condition, while E3 denotes the plan-execution divergence subset of search-space failures.

4 Answer-First and Plan-First Parallelism

4.1 Baseline: Answer-First Parallelism

Answer-First Parallelism (AF) generates K candidate solutions directly and independently from problem q:

$$a_i \sim p_\theta(a \mid q), \quad i = 1, \ldots, K$$

The selector then chooses the final answer:

$$j^* = \arg\max_i \, s(q, a_i), \qquad \hat{y}_{\mathrm{AF}}(q) = \mathrm{ans}(a_{j^*})$$

AF thus relies entirely on answer-level diversity across independently sampled solutions.

4.2 Proposed Method: Plan-First Parallelism

Plan-First Parallelism (PF) generates K distinct solution plans prior to any execution. Each plan z_i represents a fundamentally different mathematical strategy:

$$z_i \sim p_\theta(z \mid q), \quad i = 1, \ldots, K$$

Each plan then conditions an independent solution generation:

$$a_i \sim p_\theta(a \mid q, z_i), \quad i = 1, \ldots, K$$

The selector chooses from plan–solution pairs:

$$j^* = \arg\max_i \, s(q, z_i, a_i), \qquad \hat{y}_{\mathrm{PF}}(q) = \mathrm{ans}(a_{j^*})$$

Unlike AF, PF establishes strategy-level diversity before solution generation, and is thus best understood as a structured parallel search rather than generic parallel sampling.

We position PF not as a complete general-purpose solution, but as a targeted intervention for diagnosing whether hard reasoning failures are rooted in strategy-level search bottlenecks.

Rather than proposing a new search framework in full generality—such as verifier-guided tree search or adaptive compute allocation—this study focuses specifically on whether plan-level branching changes the candidate distribution, alleviates search-space failure (E1), and if so, why its gains do not always fully propagate to final accuracy.

4.3 Selector

Final answer selection proceeds in two stages. First, if two or more candidates agree on the same answer, that answer is selected by majority vote [4]. When no majority exists, an LLM verifier [16, 17] is invoked to compare the reasoning quality of all candidates and select the most logically complete solution. More robust selection strategies, such as critique-based selection and backward verification, are left as future work for mitigating E2 failures.

5 Experiments

5.1 Datasets

We evaluate on two exact-answer mathematical reasoning benchmarks: AIME and BeyondAIME. Both datasets are well-suited for our analysis, as their short integer-valued answers enable automatic grading and clean separation between candidate generation failure and selection failure. Compared to the MATH dataset used in prior work [1], both benchmarks target a substantially harder reasoning regime. AIME provides competition-style problems whose answers are integers in the range 000–999, while BeyondAIME is a curated benchmark containing problems of difficulty exceeding the later AIME problems, making it suitable for examining whether our findings generalize to even harder reasoning settings.

Starting from a raw pool of 90 AIME and 100 BeyondAIME problems, we applied a difficulty screening procedure using o3-mini under the AF condition. Each problem was solved three times independently. Problems solved correctly on all three attempts were treated as too easy, while problems solved incorrectly on all three attempts were treated as too hard to discriminate under this screening model. These problems were excluded in order to concentrate the evaluation on a hard-but-discriminative regime where differences between AF and PF are more likely to be observable.

This procedure yielded a final evaluation set of 52 AIME and 76 BeyondAIME problems, totaling 128 problems. In total, 38 AIME and 24 BeyondAIME problems were excluded by this screening. We therefore refer to this evaluation set as an *o3-mini-screened hard subset*. Because the subset was selected based on o3-mini behavior under the AF condition, it should not be interpreted as a model-neutral measure of problem difficulty. In particular, the cross-model comparisons with GPT-4.1 and Claude Haiku 4.5 should be understood as evaluations on the same o3-mini-screened hard subset, rather than as claims about each model's performance on an independently calibrated hard subset.

5.2 Experimental Setup

We fix $K = 3$ across all experiments as a practical balance between candidate diversity and computational cost, as justified by the ablation in Table 1. Final answer selection follows the two-stage selector described in Section 4.3.

We ablate $K \in \{2, 3, 4\}$ using o3-mini and find that $K = 3$ achieves the best PF accuracy on both AIME (65.4%) and BeyondAIME (55.3%). While $K = 4$ yields higher diversity scores

(Plan Diversity 4.35 vs. 3.90 at $K = 2$), it increases selection errors (E2: 17.4% on AIME, 13.2% on BeyondAIME) and degrades accuracy, suggesting that the verifier becomes less reliable as the candidate pool grows.

Table 1: Ablation on number of candidates K (o3-mini).

	AIME (52)		BeyondAIME (76)	
K	AF Acc	PF Acc	AF Acc	PF Acc
2	48.1%	59.6%	52.6%	51.3%
3	53.8%	**65.4%**	40.8%	**55.3%**
4	55.8%	55.8%	55.3%	50.0%

The non-monotonic variation of AF accuracy across K—most notably the drop to 40.8% at $K = 3$ on BeyondAIME—should be interpreted with sampling variance in mind. Each cell in Table 1 corresponds to an independent run in which K candidates are freshly sampled and the final answer is decided by majority voting over a small candidate pool, so run-to-run fluctuation is expected. On the 76-problem BeyondAIME subset, a 95% binomial confidence interval around an accuracy of 50% spans roughly ±11 percentage points; the observed AF variation across K (40.8–55.3%) therefore falls within the range attributable to sampling noise rather than indicating a systematic K-dependent trend. We accordingly treat Table 1 as practical guidance for choosing K, rather than as a precise characterization of AF scaling behavior.

We evaluate three models: **o3-mini** (OpenAI) serves as the primary model, as it is a reasoning-oriented model well-suited for hard mathematical problems. **GPT-4.1** (OpenAI) and **Claude Haiku 4.5** (Anthropic) are included as general-purpose comparison models to examine whether the effect of PF generalizes across model types.

In the PF condition, plan generation is prompted with an explicit diversity constraint requiring each of the three plans to employ a fundamentally different mathematical tool or perspective. This is intended to prevent the three plans from converging to similar strategies.

To analyze the effect of PF beyond final accuracy, we adopt three LLM-as-a-Judge metrics. **Plan Diversity** measures how strategically distinct the three generated plans are from one another. **Execution Diversity** measures how differently the actual solution paths unfold across the three candidates. **Adherence** measures how faithfully each solution follows its assigned plan [24]. The judge model is the same model used for solution generation; the possibility of self-evaluation bias is acknowledged as a limitation of this study.

Answer correctness is evaluated using exact string matching after answer extraction, as detailed in Section 5.3.

5.3 Implementation Details for Reproducibility

To improve reproducibility, we provide additional details on the prompts, decoding settings, final-answer selection procedure, LLM-as-a-Judge rubric, and answer matching rules used in our experiments. In the o3-mini experiments, no explicit temperature, top-p, or maximum-token parameters were set, and the API defaults were used. Instead, decoding effort was controlled using the `reasoning_effort` parameter: `medium` for AF candidate generation, `low` for PF plan generation, `high` for PF solution execution, and `low` for verifier and LLM-as-a-Judge calls. For the comparison-model runs, we used the same prompting, selection, and evaluation pipeline,

with model-specific API defaults unless otherwise required. No external tools, retrieval modules, or calculators were used.

Prompts. In the AF condition, each model was directly asked to solve the problem step by step and return a final answer enclosed in `\boxed{}`. In the PF condition, the model first generated three distinct solution plans using an explicit diversity instruction, and each plan was then executed independently. The PF execution prompt required the model to begin from the assigned initial angle of attack, while allowing it to pivot if the direction became blocked. The AF, PF plan-generation, PF execution, verifier, and judge prompts are summarized here rather than reproduced in full due to space constraints.

Selection and judging. Final answer selection was performed by majority voting when at least two candidates produced the same extracted answer. When no majority existed, an LLM verifier compared the three candidate solutions and selected the attempt with the most logically sound reasoning by returning a single index. For PF outputs, Plan Diversity, Execution Diversity, and Adherence were scored on a 1–5 scale using an LLM-as-a-Judge prompt.

Answer extraction and matching. Final answers were extracted from the model output by identifying answers enclosed in the `\boxed{...}` pattern. If no boxed answer was found, the extracted answer was treated as empty. After extraction, surrounding whitespace was removed, and correctness was evaluated by exact string matching against the gold answer. We did not apply additional numeric normalization or zero-padding. Therefore, AIME-style leading-zero variants such as `72` and `072` could be treated as different strings. This is a limitation of the evaluation protocol, but the same rule was applied to both AF and PF outputs.

5.4 Results

5.4.1 o3-mini (Primary Model)

On the AIME benchmark, PF achieves a CandHit of 71.2% (37/52) and an accuracy of 65.4% (34/52), compared to 65.4% (34/52) and 53.8% (28/52) for AF, representing gains of +5.8%p in CandHit and +11.5%p in accuracy. This indicates that PF not only secured more valid candidate solutions at the search stage, but that these gains translated substantially into final answer selection.

Among the 18 incorrect PF responses on AIME, 15 (83.3%) are classified as E1 (Search-space failure), 3 (16.7%) as E2 (Selection failure), and 0 as E3 (Plan-execution divergence). On BeyondAIME, among 34 incorrect responses, 26 (76.5%) are E1, 6 (17.6%) are E2, and 2 (5.9%) are E3. Across both datasets, E1 consistently dominates, suggesting that even with plan-level branching, the search space is not sufficiently opened for the hardest problems.

LLM-as-a-Judge scores reveal a consistent gap between plan diversity (3.63/5.0) and execution diversity (2.77/5.0), despite high adherence (4.06/5.0). This indicates that while the model generates strategically distinct plans and largely follows them, the actual solution trajectories do not diverge as much as the plans suggest.

On BeyondAIME, PF shows even larger gains, improving CandHit by +13.2%p (50.0% $\rightarrow$ 63.2%) and accuracy by +14.5%p (40.8% $\rightarrow$ 55.3%). The error breakdown shows E1 at 76.5%, E2 at 17.6%, and E3 at 5.9%, with the plan–execution diversity gap again present (Plan Diversity 3.89 vs. Execution Diversity 2.93).

Taken together, these results suggest that PF tends to yield larger gains on harder problems, where answer-level diversity alone is less likely to cover the correct solution space.

5.4.2 Comparison Models

For GPT-4.1, PF yields no consistent improvement. On AIME, accuracy slightly decreases from 26.9% to 23.1%, and on BeyondAIME it increases marginally from 17.1% to 22.4%, while CandHit declines. Among the 40 incorrect PF responses on AIME, 33 (82.5%) are E1, 4 (10.0%) are E2, and 3 (7.5%) are E3. On BeyondAIME, among 59 incorrect responses, 54 (91.5%) are E1 and 5 (8.5%) are E2. E1 thus accounts for 82.5–91.5% of all PF errors, indicating that the primary bottleneck is the inability to generate correct candidates regardless of planning structure. Notably, GPT-4.1 achieves the highest Plan Diversity scores (4.71–4.79) among all models, yet this does not translate into improved performance.

For Claude Haiku 4.5, PF produces a small accuracy gain on AIME (+3.8%p) but a slight decrease on BeyondAIME (−2.6%p). Among the 37 incorrect PF responses on AIME, 21 (56.8%) are E1, 3 (8.1%) are E2, and 13 (35.1%) are E3. On BeyondAIME, among 61 incorrect responses, 28 (45.9%) are E1, 8 (13.1%) are E2, and 25 (41.0%) are E3. E3 (Plan-execution divergence) thus accounts for 35.1% and 41.0% of PF errors on AIME and BeyondAIME respectively—by far the highest among all models. Adherence scores are also the lowest (2.82–3.06), suggesting that the model frequently fails to follow its assigned plans and instead converges to similar solution paths regardless of the stated strategy.

Both comparison models show limited and inconsistent benefits from PF, but for qualitatively different reasons. A detailed analysis of these failure patterns is provided in Section 6.

Table 2: Comparison of AF and PF across models and datasets on the o3-mini-screened hard subset. CandHit denotes the proportion of problems for which at least one correct candidate was generated, and Acc denotes final answer accuracy.

Model	Dataset	AF CandHit	AF Acc	PF CandHit	PF Acc
o3-mini	AIME	65.4%	53.8%	71.2%	**65.4%**
o3-mini	BeyondAIME	50.0%	40.8%	63.2%	**55.3%**
GPT-4.1	AIME	30.8%	26.9%	30.8%	23.1%
GPT-4.1	BeyondAIME	31.6%	17.1%	28.9%	22.4%
Claude Haiku 4.5	AIME	32.7%	25.0%	34.6%	28.8%
Claude Haiku 4.5	BeyondAIME	26.3%	22.4%	30.3%	19.7%

Table 3: Mutually exclusive error breakdown and LLM-as-a-Judge scores for PF across models and datasets. E3 denotes plan-execution divergence.

Model	Dataset	E1	E2	E3	Plan Div.	Exec Div.	Adh.
o3-mini	AIME	83.3%	16.7%	–	3.63	2.77	4.06
o3-mini	BeyondAIME	76.5%	17.6%	5.9%	3.89	2.93	4.18
GPT-4.1	AIME	82.5%	10.0%	7.5%	4.71	3.87	4.75
GPT-4.1	BeyondAIME	91.5%	8.5%	–	4.79	4.26	4.80
Claude Haiku 4.5	AIME	56.8%	8.1%	35.1%	3.31	2.17	3.06
Claude Haiku 4.5	BeyondAIME	45.9%	13.1%	41.0%	3.54	2.22	2.82

Table 4: Sensitivity analysis of E3 rates under alternative plan-execution divergence thresholds.

Model	Dataset	Strict ($PD \geq 5$, $ED \leq 1$)	Original ($PD \geq 4$, $ED \leq 2$)	Loose ($PD \geq 3$, $ED \leq 3$)
o3-mini	AIME	0 (0.0%)	0 (0.0%)	10 (55.6%)
o3-mini	BeyondAIME	0 (0.0%)	2 (5.9%)	18 (52.9%)
GPT-4.1	AIME	0 (0.0%)	3 (7.5%)	7 (17.5%)
GPT-4.1	BeyondAIME	0 (0.0%)	0 (0.0%)	2 (3.4%)
Claude Haiku 4.5	AIME	1 (2.7%)	13 (35.1%)	24 (64.9%)
Claude Haiku 4.5	BeyondAIME	0 (0.0%)	25 (41.0%)	38 (62.3%)

Note. PD: Plan Diversity, ED: Execution Diversity. Percentages are computed over PF incorrect cases.

The original threshold is not intended to be optimal in a supervised sense, as no ground-truth E3 labels are available. Instead, it serves as a conservative and interpretable operating definition between overly strict and overly loose criteria. As expected, the strict setting identifies very few E3 cases, while the loose setting captures a broader set of divergence-like failures. The sensitivity results show that E3 counts are more sensitive to the Execution Diversity cutoff than to the Plan Diversity cutoff. Under the original threshold, Claude Haiku 4.5 exhibits substantially higher E3 rates than the other models, and this pattern remains visible under the loose setting. This suggests that the observed plan-execution divergence pattern is not merely an artifact of a single threshold choice.

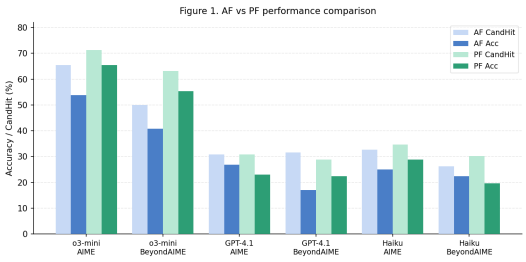


Figure 1: AF vs PF performance comparison. CandHit: proportion of problems with at least one correct candidate; Acc: final answer accuracy.

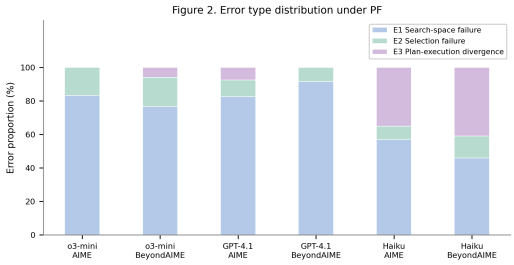


Figure 2: Error type distribution under PF. E1: search-space failure, E2: selection failure, E3: plan-execution divergence.

6 Discussion

6.1 Effectiveness of Plan-First for o3-mini: Adherence and Search Bottlenecks

On the AIME benchmark, PF outperformed AF by +5.8%p in Hit Rate and +11.5%p in Accuracy for o3-mini; on BeyondAIME, the gains were +13.2%p in Hit Rate and +14.5%p in Accuracy. In other words, PF yielded consistent improvements at both the search stage and the final selection stage for o3-mini.

o3-mini also recorded Adherence scores of 4.06 and 4.18, indicating that the model largely followed the generated plans throughout the solution process. This suggests that plan-following—the core premise of PF—holds relatively stably for this model. In other words, o3-mini is capable not only of generating plans, but of translating them into actual reasoning trajectories.

Nonetheless, Execution Diversity (2.77/2.93) was consistently lower than Plan Diversity (3.63/3.89), suggesting that plan-level diversity is not fully preserved during execution. That said, the co-occurrence of high Adherence and substantial performance gains indicates that this loss was not severe enough to undermine the effectiveness of PF for o3-mini.

The error analysis further supports this interpretation. E2 accounted for only 16.7% and 17.6% of failures, indicating that the selection mechanism functioned relatively well, whereas E1 remained the dominant failure type at 83.3% and 76.5% respectively. This suggests that even for o3-mini, the core bottleneck lies in search rather than selection. Thus, while PF meaningfully improved o3-mini's performance, the hardest problems still frequently defied correct strategy generation, and further expanding the search space remains an open challenge for future work.

6.2 Limitations of Plan-First for GPT-4.1: Plan Diversity Without Strategic Validity

PF did not yield consistent gains for GPT-4.1. On AIME, PF reduced Accuracy by 3.8%p relative to AF, with no change in Hit Rate. On BeyondAIME, Accuracy improved by 5.3%p, but Hit Rate decreased by 2.6%p. Overall, PF cannot be said to have reliably improved GPT-4.1's search performance.

These results highlight a disconnect between generating diverse plans and actually opening a search space that contains correct strategies. GPT-4.1 achieved the highest Plan Diversity scores among all models (4.71/4.79), yet the majority of PF errors still fell under E1 (82.5%, 91.5%). This indicates that generating diverse plans does not translate into performance gains unless those plans include strategies that can lead to correct answers.

GPT-4.1 also showed relatively high Execution Diversity and Adherence. The core issue, therefore, is not that the model failed to execute its plans, but rather that the executed plans, despite being diverse, all led away from correct answers. In other words, GPT-4.1's limitation lies not in plan-following, but in its inability to place valid strategies within the search space on hard reasoning problems.

This suggests that PF's effectiveness requires not only the ability to generate diverse plans, but also the reasoning quality to ensure that the search space contains strategies with a genuine chance of being correct.

6.3 Plan-Execution Divergence in Claude Haiku: Adherence Failure as a Distinct Bottleneck

PF also failed to produce consistent gains for Claude Haiku 4.5. On AIME, PF improved Accuracy by 3.8%p and Hit Rate by 1.9%p. On BeyondAIME, however, Hit Rate increased by 3.9%p while Accuracy declined by 2.7%p. Thus, while PF contributed to generating more correct candidates at the search stage, these gains did not reliably carry through to final answer selection.

The most striking characteristic is the proportion of E3 errors: 35.1% on AIME and 41.0% on BeyondAIME—by far the highest among all models. While other models showed little to no E3, Claude Haiku exhibited a frequent pattern in which diverse plans failed to be maintained during execution. This demonstrates that PF's failure modes extend beyond search-space failure

(E1) and selection failure (E2) to include plan-execution divergence as a distinct and significant bottleneck.

This interpretation is consistent with the model's low Adherence scores (3.06 and 2.82)—the lowest among all models—indicating that generated plans were not sufficiently maintained during the solution process. In other words, rather than following its assigned strategy, the model tended to converge toward similar incorrect solution paths regardless of the plan.

This implies that PF's effectiveness requires not only the ability to generate plans, but also the instruction-following capability to stably translate those plans into actual reasoning trajectories. This behavior reflects a lack of reasoning resilience — the ability of a model to maintain its intended strategy under the complexity of execution. From the perspective of resilient AI system design, a model that generates valid plans but cannot sustain them offers limited reliability in practice.

6.4 Cross-Model Synthesis

Taken together, the results across three models show that PF is not a strategy that uniformly improves performance simply by generating diverse plans. It was effective for o3-mini, which can translate plans into actual solutions, but ineffective for GPT-4.1, which despite generating highly diverse plans failed to identify correct strategies. For Claude Haiku, plan-level diversity was frequently not preserved during execution.

Therefore, the effectiveness of PF depends not on plan diversity alone, but on the combination of reasoning quality and instruction-following capability.

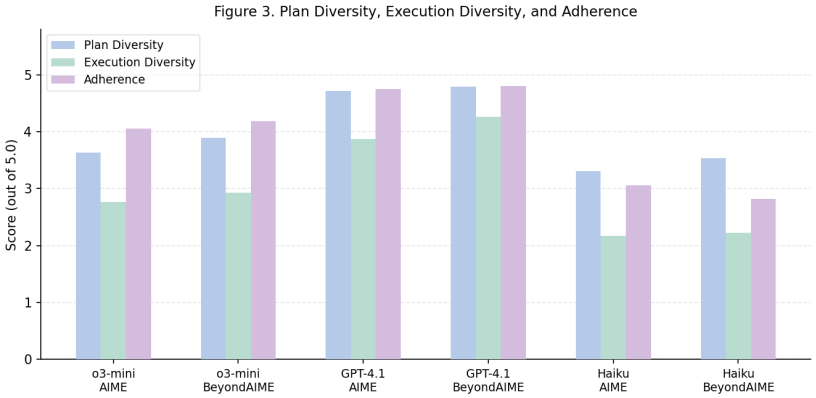


Figure 3: Plan Diversity, Execution Diversity, and Adherence scores under PF across models and datasets (LLM-as-a-Judge, out of 5.0).

6.5 Implications for E-Business Information Systems and Cyber Resilience

Within the scope of EBISION 2026, this work most directly addresses the topics of "Artificial Intelligence and Data-Driven Business Innovation," "Resilient and Adaptive EBIS Architectures," and "Cybersecurity and Digital Trust in EBIS." It analyzes the failure structure of LLM reasoning components that increasingly underpin AI-driven decision support in e-business information systems (Section 1), and interprets Plan-First Parallelism as a strategy-level mechanism that

contributes to the cyber resilience of such systems by making reasoning failures diagnosable and recoverable [27] (this section and Section 7).

The proposed taxonomy also has implications for LLM-enabled e-business information systems. In enterprise settings, LLMs may support decision-making, compliance analysis, financial reasoning, operational planning, and incident triage. For such systems, reliability requires more than high average accuracy: the system should make failures diagnosable, support recovery mechanisms, and avoid overreliance on a single reasoning trajectory. We do not claim that mathematical benchmarks directly instantiate e-business workflows; rather, they provide a controlled stress-test for reasoning failures that may also occur in LLM-enabled enterprise systems.

Our E1–E3 taxonomy maps naturally to different failure-handling strategies. E1 indicates that no correct candidate was generated, suggesting the need for broader search, model routing, retrieval augmentation, or human escalation. E2 indicates that a correct candidate existed but was not selected, suggesting the need for stronger verification, cross-model judging, or improved answer normalization. E3 indicates that plan-level diversity was not preserved during execution, suggesting the need for adherence monitoring and execution-level diversity control. These distinctions are important for cyber-resilient AI systems because different failure modes require different recovery mechanisms.

From this perspective, Plan-First Parallelism can be interpreted as a strategy-level resilience mechanism. By maintaining multiple planned reasoning trajectories, PF reduces dependence on a single solution path. However, our cross-model results show that plan diversity alone is insufficient: reliable deployment also requires valid strategy generation and stable plan execution. This finding is directly relevant to e-business information systems in which LLM outputs may influence organizational decisions, risk assessments, or automated workflows.

7 Conclusion

This study investigates whether generating solution plans prior to execution improves hard mathematical reasoning under a fixed candidate budget. We propose Plan-First Parallelism (PF), a strategy-level branching approach that generates multiple distinct solution plans before execution, and compare it against Answer-First Parallelism (AF) across two benchmarks and three models. To support systematic failure analysis, we introduce a three-way error taxonomy distinguishing search-space failure (E1), selection failure (E2), and plan-execution divergence (E3).

Our primary findings are as follows. First, PF consistently outperforms AF on both AIME and BeyondAIME when using o3-mini, improving CandHit by up to +13.2%p and accuracy by up to +14.5%p. Second, error analysis reveals that E1 (Search-space failure) remains the dominant bottleneck across all models, indicating that generating correct candidates is fundamentally harder than selecting among them. Third, LLM-as-a-Judge scores consistently show a gap between Plan Diversity and Execution Diversity, suggesting that strategically distinct plans do not always produce correspondingly distinct solution trajectories. Fourth, Claude Haiku 4.5 exhibits a high rate of E3 (Plan-execution divergence), revealing that plan-execution divergence is a distinct failure mode that varies meaningfully across model types.

Taken together, these results suggest that the effectiveness of PF is not a function of plan diversity alone, but depends critically on two complementary capabilities: the reasoning quality required to place valid strategies within the search space, and the instruction-following capability required to translate those strategies into actual solution trajectories. Models that lack either capability fail under PF for qualitatively different reasons—GPT-4.1 generates diverse

but strategically invalid plans, while Claude Haiku fails to sustain its plans during execution. This cross-model contrast highlights that plan-level branching is a necessary but not sufficient condition for improved reasoning performance.

This study has several limitations. The evaluation set is relatively small (128 problems), and it is an o3-mini-screened hard subset rather than a model-neutral difficulty split. Since the subset was selected based on o3-mini behavior under the AF condition, the cross-model comparisons should be interpreted with this selection bias in mind. The range of models compared is also limited.

Future work should explore alternative structured search strategies beyond PF, and investigate more robust selection mechanisms to mitigate E2 failures. Improving execution diversity—the gap between plan-level and solution-level diversity—also remains an open challenge. Additionally, the results suggest a promising hybrid direction: using o3-mini for plan generation, where it demonstrates strong reasoning ability, and GPT-4.1 as a verifier or judge, where it achieves high diversity scores. Such a cross-model pipeline may yield stronger performance than any single-model configuration, and represents a natural next step toward more robust and reliable reasoning systems.

Beyond benchmark-level findings, this work contributes to the broader design of reliable and cyber-resilient AI components for e-business information systems. By decomposing LLM reasoning failures into search-space failure, selection failure, and plan-execution divergence, the proposed framework makes failures more diagnosable and actionable. Future work should extend this failure-aware analysis from controlled mathematical benchmarks to enterprise decision support, compliance reasoning, cyber-risk analysis, and other e-business information system scenarios where reliable LLM reasoning is required.

Acknowledgments

This research was supported by the MSIT (Ministry of Science and ICT), Korea, under the Graduate School of Virtual Convergence support program (IITP-2026-RS-2023-00254129) supervised by the IITP (Institute for Information & Communications Technology Planning & Evaluation). This research was also supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2024-00439139, Development of a Cyber Crisis Response and Resilience Test Evaluation Systems).

References

[1] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., & Steinhardt, J. (2021). Measuring mathematical problem solving with the MATH dataset. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.

[2] Snell, C., Lee, J., Xu, K., & Kumar, A. (2025). Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *The Thirteenth International Conference on Learning Representations*.

[3] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*.

[4] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. *The Eleventh International Conference on Learning Representations*.

[5] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., & Lim, E.-P. (2023). Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.

[6] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*.

[7] Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., Schuurmans, D., Cui, C., Bousquet, O., Le, Q., & Chi, E. (2023). Least-to-most prompting enables complex reasoning in large language models. *The Eleventh International Conference on Learning Representations*.

[8] Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., & Neubig, G. (2023). PAL: Program-aided language models. *Proceedings of the 40th International Conference on Machine Learning*.

[9] Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*.

[10] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*.

[11] Zelikman, E., Wu, Y., Mu, J., & Goodman, N. (2022). STaR: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*.

[12] Zhang, Z., Zhang, A., Li, M., & Smola, A. (2023). Automatic chain of thought prompting in large language models. *The Eleventh International Conference on Learning Representations*.

[13] Khot, T., Trivedi, H., Finlayson, M., Fu, Y., Richardson, K., Clark, P., & Sabharwal, A. (2023). Decomposed prompting: A modular approach for solving complex tasks. *The Eleventh International Conference on Learning Representations*.

[14] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Welleck, S., Majumder, B. P., Gupta, S., Hajishirzi, H., Choi, Y., & Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*.

[15] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*.

[16] Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2024). Let's verify step by step. *The Twelfth International Conference on Learning Representations*.

[17] Wang, P., Li, L., Shao, Z., Xu, R., Dai, D., Li, Y., Chen, D., Wu, Y., & Sui, Z. (2024). Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

[18] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*.

[19] Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*.

[20] Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature, 529*, 484–489.

[21] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., et al. (2017). Mastering the game of Go without human knowledge. *Nature, 550*, 354–359.

[22] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *The Eleventh International Conference on Learning*

Representations.

[23] Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., Li, Y., Wang, X., Dehghani, M., Brahma, S., et al. (2024). Scaling instruction-finetuned language models. *Journal of Machine Learning Research, 25*(70), 1–53.

[24] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*.

[25] Sun, T., Yao, Y., Wang, X., & Chen, J. (2025). Challenging the boundaries of reasoning: An olympiad-level math benchmark for large language models. *arXiv preprint arXiv:2503.21380*.

[26] ByteDance Seed, Chen, J., Fan, T., Liu, X., Liu, L., Lin, Z., Wang, M., et al. (2025). Seed1.5-Thinking: Advancing superb reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.13914*.

[27] Ross, R., Pillitteri, V., Graubart, R., Bodeau, D., & McQuaid, R. (2021). Developing cyber-resilient systems: A systems security engineering approach. *NIST Special Publication 800-160, Volume 2, Revision 1*. National Institute of Standards and Technology.